DOCUMENTATION TECHNIQUE

Documentation technique — Concert Agent

Architecture, tools, base SQLite, sécurité, tests et choix techniques.

CA

Concert Agent · Octobre 2026

1. Objectif

Concert Agent est une application Flask qui répond à une demande en langage naturel en choisissant et en exécutant des **tools** Python. Les données de concerts viennent d'une API réelle (Ticketmaster) ou, à défaut, d'un jeu de démonstration étiqueté. L'agent ne fabrique jamais de date, de salle ou de prix.

Le projet fusionne aussi l'ancien site Spotify : connexion OAuth, profil et import des artistes écoutés dans SQLite, sans second backend Node.

Le projet est volontairement séparé en couches :



---

2. Cycle d'une demande

1. Le frontend envoie `POST /api/chat` avec `{ "message": "..." }`.
2. `app.py` récupère le `session_id` Flask et appelle `run_agent`.
3. `run_agent` vérifie d'abord s'il existe une action en attente de confirmation.
4. Sinon, il construit l'historique récent et appelle le LLM avec le prompt système et les tools.
5. Le LLM renvoie soit du texte, soit un ou plusieurs `tool_calls`.
6. Pour chaque tool call :
   - le JSON est décodé ;
   - le tool est retrouvé dans `TOOL_MAP` ;
   - ses arguments sont validés ;
   - s'il est en lecture, il est exécuté et son résultat est renvoyé au LLM ;
   - s'il est en écriture, l'action est stockée dans `pending_actions` et la boucle s'arrête pour demander confirmation.
7. Le LLM produit la réponse finale à partir des résultats réels.
8. La demande, la réponse et les traces sont enregistrées dans SQLite.

Sans clé LLM, ou en cas d'erreur LLM, `agent/regles.py` exécute les mêmes tools avec des règles déterministes. Le frontend affiche alors « mode règles ».

3. Tools

Tool	Type	Rôle
`consulter_preferences`	lecture	Lit la ville, la région, le budget et les artistes suivis.
`rechercher_concerts`	lecture	Cherche via Ticketmaster, filtre par artiste, ville, région, dates et budget.
`lister_favoris`	lecture	Liste les concerts enregistrés.
`historique_agent`	lecture	Retourne les dernières interactions et traces.
`ajouter_favori`	écriture	Ajoute un concert de la dernière recherche aux favoris.
`ajouter_artiste_suivi`	écriture	Ajoute un artiste à la liste suivie.

Chaque tool possède une description en français dans `tools.py`. C'est cette description qui est envoyée au LLM dans le schéma OpenAI `tools`.

Le tool `rechercher_concerts` renvoie aussi un champ `affichage` déjà formaté. L'agent reprend ce bloc dans sa réponse finale : chaque concert affiche l'artiste, la date, l'heure, le lieu, le prix et l'URL de billetterie. Le frontend transforme les URL en liens cliquables.

Validation des écritures

`ajouter_favori` n'accepte pas un concert arbitraire : il vérifie que le `concert_id` provient du cache de la dernière recherche de la session. Cela empêche le LLM d'inventer un identifiant.

`ajouter_artiste_suivi` vérifie que le nom n'est pas vide.

4. Confirmation avant écriture

Séquence :

```
Utilisateur : Ajoute le concert de Gims à Décines à mes favoris.
Agent       : Je vais ajouter « Gims – 02/11/2026 à Décines (LDLC Arena) » à tes favoris. Confirmer ?
Utilisateur : Oui
Agent       : ✅ C'est fait : ... a été ajouté à tes favoris (n° 1).
```

- L'action est stockée dans `pending_actions` avec le `session_id`.
- Aucune écriture n'a lieu avant le « Oui ».
- « Non » supprime l'action et écrit une trace `refuse`.
- Une réponse « Oui » est interprétée par `_verdict` (oui, ok, confirme, valide, go...).
- Le bouton « Oui » du frontend envoie `{ "confirm": "oui" }`.

5. Base SQLite

Fichier : `data/concerts.db` (ignoré par Git).

Table	Contenu
`preferences`	Une ligne : ville, région, code région, budget.
`artistes_suivis`	Artistes suivis, uniques.
`favoris`	Concerts enregistrés, `concert_id` unique.
`interactions`	Messages user/assistant par session.
`tool_traces`	Tool, arguments, résultat, statut, durée.
`pending_actions`	Action d'écriture en attente de confirmation.
`resultats_cache`	Derniers concerts trouvés, pour valider `ajouter_favori`.
`concerts_demo`	Jeu de démonstration recalculé au démarrage.
`spotify_sessions`	Jetons Spotify, expiration et profil, par session Flask.

Le schéma est créé par `db.init_db()` au démarrage. Le seed ajoute 9 artistes, des préférences Lyon / Auvergne-Rhône-Alpes / 60 € et 12 concerts de démonstration.

6. LLM

`agent/llm.py` est le **seul** endroit qui appelle un modèle. Le client utilise une API compatible OpenAI :

```
POST {LLM_BASE_URL}/chat/completions
```

Configuration :

```
LLM_PROVIDER=gemini
LLM_MODEL=gemini-3.5-flash-lite
LLM_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai
GEMINI_API_KEY=...
```

Providers supportés par le code : Gemini, OpenAI, OpenRouter, Groq, Ollama.

Le client :

- envoie `tools` et `tool_choice="auto"` ;
- normalise les `tool_calls` (arguments en chaîne JSON, identifiants manquants) ;
- convertit `function_call` en `tool_calls` pour les anciens modèles ;
- lève `LLMError` en cas d'erreur HTTP ou de réponse illisible.

Prompt système

Le prompt interdit explicitement d'inventer des concerts et impose l'utilisation des tools pour toute demande de concerts. Il précise aussi que les tools d'écriture sont confirmés automatiquement par le système.

Secours

Si `obtenir_llm()` renvoie `None` (pas de clé) ou si le LLM lève `LLMError`, `run_rules` prend le relais. Le mode est indiqué dans la réponse (`regles` ou `regles_fallback`) et dans le badge de l'interface.

---

7. Intégration Spotify

L'ancien site Node est remplacé par une intégration directe dans Flask :

1. `GET /login` génère un `state` aléatoire, le stocke dans la session signée Flask et redirige vers Spotify.
2. `GET /callback` vérifie le `state`, échange le code contre un jeton et récupère le profil.
3. Les jetons sont enregistrés dans `spotify_sessions` (SQLite), jamais dans le cookie.
4. Les top artistes et les écoutes récentes sont fusionnés, dédupliqués et ajoutés à `artistes_suivis`.
5. `GET /logout` supprime la session Spotify côté serveur.

Scopes demandés : `user-read-private`, `user-top-read`, `user-read-recently-played`.

```
SPOTIFY_CLIENT_ID=...
SPOTIFY_CLIENT_SECRET=...
REDIRECT_URI=http://127.0.0.1:5000/callback
```

L'URI doit être strictement identique à celle du dashboard Spotify. Spotify refuse désormais l'alias `localhost` : en local, utiliser l'adresse loopback explicite `http://127.0.0.1:5000/callback`. En production, utiliser `https://<service>.onrender.com/callback`. En mode développement, Spotify limite l'accès à 5 comptes ajoutés dans **User Management**.

Le refresh token est conservé quand Spotify n'en renvoie pas lors d'un rafraîchissement. Si le refresh échoue, la session est supprimée et l'utilisateur doit se reconnecter. L'import est limité à 30 artistes pour ne pas saturer la barre latérale.

8. API HTTP

Méthode	Route	Rôle
`GET`	`/`	Interface.
`POST`	`/api/chat`	`{message}` ou `{confirm:"oui"/"non"}`.
`GET`	`/api/historique`	Interactions et traces de la session.
`GET`	`/api/preferences`	Préférences, artistes, favoris.
`POST`	`/api/preferences`	Met à jour ville, région, budget.
`GET`	`/api/artistes`	Liste des artistes suivis.
`POST`	`/api/artistes`	Ajoute un artiste.
`DELETE`	`/api/artistes/<nom>`	Retire un artiste.
`GET`	`/api/favoris`	Liste des favoris.
`DELETE`	`/api/favoris/<id>`	Retire un favori.
`GET`	`/api/etat`	État LLM, Ticketmaster, Spotify, pending, date.
`GET`	`/login`	Redirection OAuth Spotify.
`GET`	`/callback`	Retour OAuth, stockage des jetons et import.
`GET`	`/logout`	Déconnexion Spotify.
`GET`	`/api/spotify/status`	Configuration, connexion et profil Spotify.
`GET`	`/api/me`	Compatibilité ancien frontend Spotify.
`POST`	`/api/spotify/import`	Importe les artistes Spotify (`{limit:30}`).
`GET`	`/api/spotify/top-artists`	Top artistes Spotify de l'utilisateur.

Réponse type de `/api/chat` :


```
{
  "reply": "J'ai trouvé 3 concert(s) : ...",
  "traces": [
    {"tool": "rechercher_concerts", "arguments": {}, "statut": "succes", "duree_ms": 12, "resume": "3
concert(s), source=demo"}
  ],
  "confirmation": null,
  "mode": "regles"
}
```

9. Données de démonstration

Le jeu de démonstration contient notamment :

- Gims à Lyon (Halle Tony Garnier), le mois prochain ;
- Sofiane Pamart à Grenoble (Summum), le mois prochain ;
- Ninho à Lyon (Halle Tony Garnier), le mois prochain ;
- plusieurs concerts à Paris, Lille, Marseille et Toulouse à des dates décalées.

Les dates sont recalculées à chaque démarrage (`_next_month_day` , `_shift`) pour que la démo reste cohérente quelle que soit la date. Toute réponse issue de ce jeu contient `source="demo"` et un avertissement visible.

10. Tests

```
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Les tests utilisent une base temporaire (`CONCERT_DB_PATH`) et forcent `MODE_DEMO=1` pour ne pas dépendre du réseau. Ils vérifient :

- la présence des 6 tools et des 2 tools d'écriture ;
- le mode règles sans clé LLM ;
- le scénario 1 (3 concerts en Auvergne-Rhône-Alpes) ;
- le scénario 2 (aucun résultat, aucune invention) ;
- le scénario 3 (confirmation puis ajout en base) ;
- le refus de confirmation ;
- l'historique et les traces ;
- l'effacement du budget ;
- les routes artistes et favoris ;
- Spotify non configuré, redirection OAuth, état invalide et import refusé ;
- import Spotify réussi, déduplication et ajout dans les artistes suivis ;
- stockage, conservation du refresh token et suppression de session Spotify.

Résultat attendu : `Ran 13 tests ... OK` .

11. Sécurité et limites

- Les secrets sont dans `.env` , ignoré par Git. Ne jamais committer `.env` .
- `FLASK_SECRET` doit être changé pour autre chose qu'une valeur de démonstration.
- Les concerts de démonstration ne sont pas réels.

- La couverture Ticketmaster en France est partielle. La recherche utilise `locale=fr-fr` puis l'identifiant exact de l'attraction pour éviter les homonymes ; si aucun événement n'est trouvé, les données de démonstration sont affichées et signalées.
- Spotify : les jetons sont en base, pas dans le cookie ; le `state` OAuth est vérifié ; les scopes restent limités au profil, aux tops et aux écoutes récentes.
- Le mode développement Spotify est limité à 5 comptes ; les autres recevront une erreur 403.
- L'application écoute sur `127.0.0.1` en local ; `HOST=0.0.0.0` est prévu pour Render/Gunicorn.
- Le LLM réel a été validé avec `gemini-3.5-flash-lite` ; il nécessite une clé Gemini et un redémarrage du serveur. Ce modèle est préféré à `gemini-3.6-flash`, dont le quota gratuit de 20 requêtes/jour est trop vite épuisé.
- La base SQLite Render gratuite est éphémère ; utiliser un disque persistant pour conserver les données.